

IoT mikroservisi zasnovani na događajima – uporedna evaluacija MQTT-a i Kafke

Student: Svetislav Veljkovic 19083

1. Uvod i arhitektura sistema

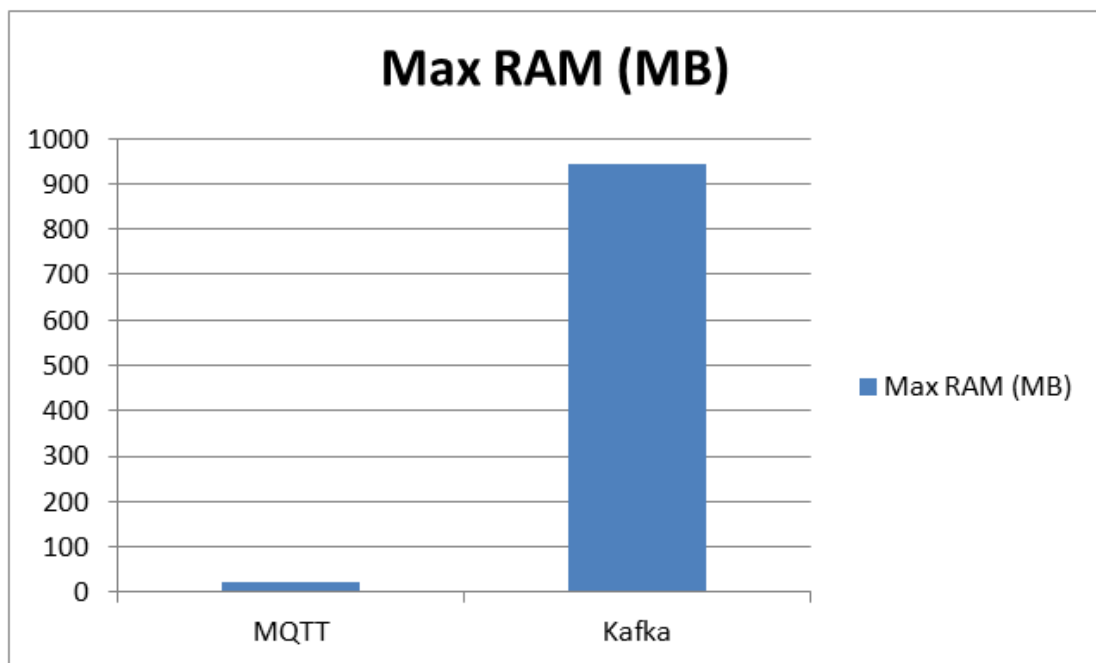
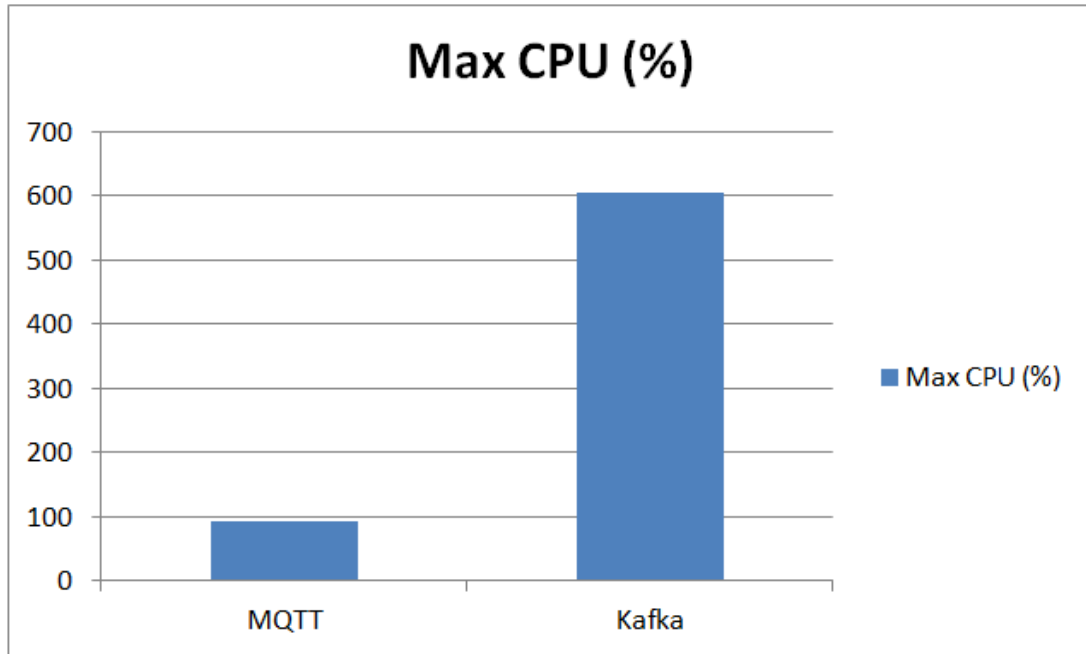
Cilj ovog projekta je uporedna evaluacija dva dominantna message broker sistema — MQTT-a (Mosquitto) i Apache Kafke — unutar event-driven IoT mikroservisne arhitekture. Sistem je dizajniran da obradjuje velike kolicine senzorskih podataka, simulirajuci realno IoT okruženje.

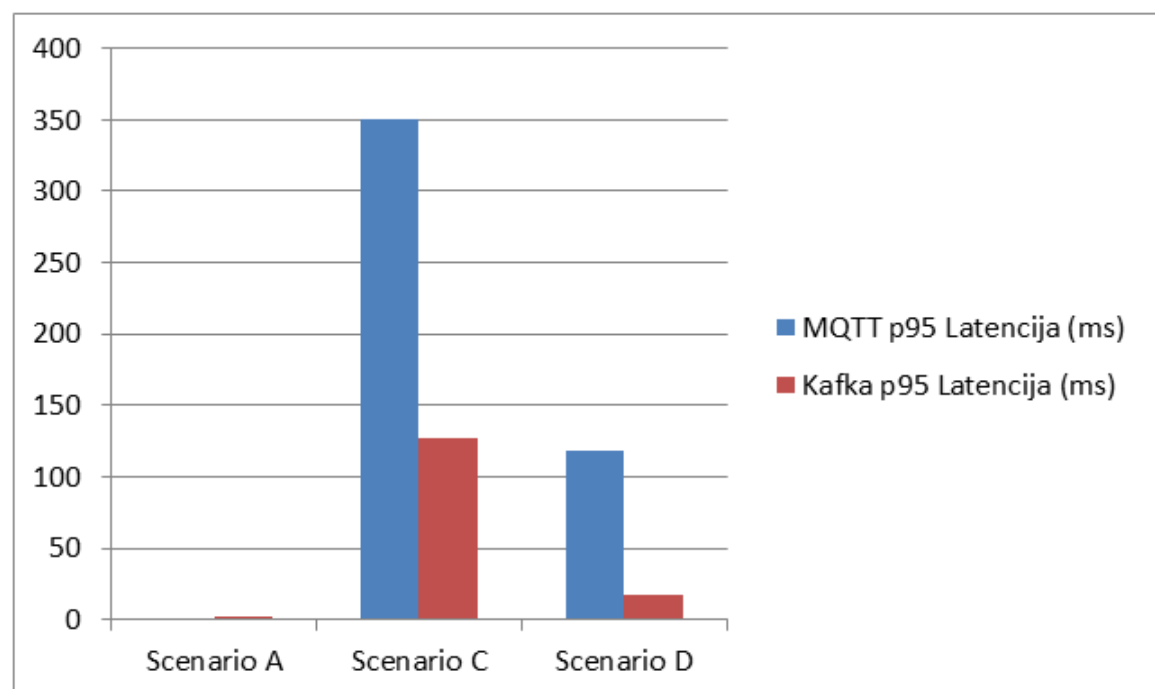
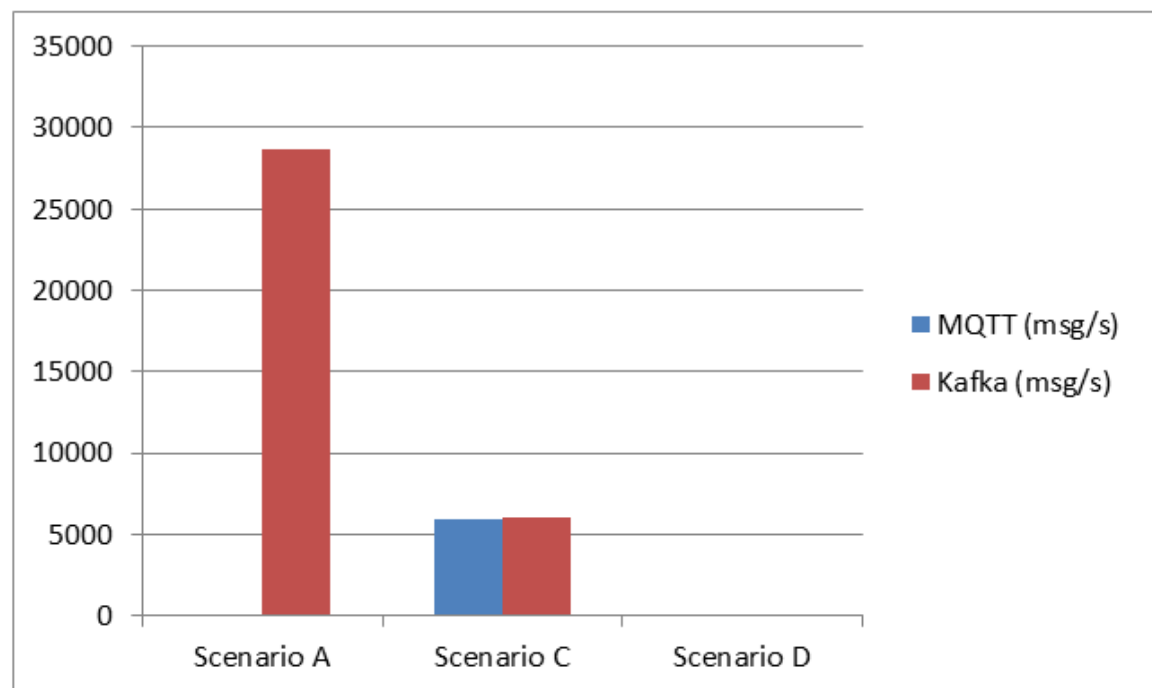
Implementirana arhitektura se sastoji od sledećih mikroservisa, potpuno kontejnerizovanih korišćenjem Docker Compose-a:

- Data Ingestion Service:** Generise podatke na osnovu realnog IoT dataseta (bez upotrebe eksternih biblioteka poput Pandas-a radi optimizacije resursa) i strimuje ih ka brokerima.
- Data Storage Service:** Mikroservis zadužen za perzistenciju podataka u PostgreSQL bazu. Implementiran je mehanizam batching-a (grupni upis) kako bi se sprečilo usko grlo (bottleneck) na I/O podsistemu tokom stress-testova.
- Analytics Service:** Servis za obradu toka podataka (Stream Processing) koji koristi Tumbling Window od 10 sekundi za racunanje prosečne temperature i generisanje kritičnih alarma ukoliko prosek pređe prag od 50°C.

2. Tabela performansi

Scenario	Broker	Throughput (msg/s)	p95 Latencija (ms)	Max CPU (%)	Max RAM (MB)
A (Massive Sensor Ingestion)	MQTT	~95	1.10	92.06%	21.8
	Kafka	~28,708	2.21	603.72%	944.0
B (Edge Connectivity Failures)	MQTT	~0 (prekid)	N/A	3.47%	903.9
	Kafka	~0 (prekid)	N/A	13.30%	21.8
C (Burst Event Load)	MQTT	~5,934	350.68	25.81%	21.8
	Kafka	~6,095	126.78	504.07%	944.0
D (Real-Time Alerting)	MQTT	~8	118.68	3.42%	28.5
	Kafka	~35	16.92	12.05%	29.0





3. Analiza kvaliteta usluge (QoS) i garancije isporuke

MQTT (Mosquitto): Tokom eksperimenata testiran je uticaj Quality of Service (QoS) nivoa na latenciju i pouzdanost:

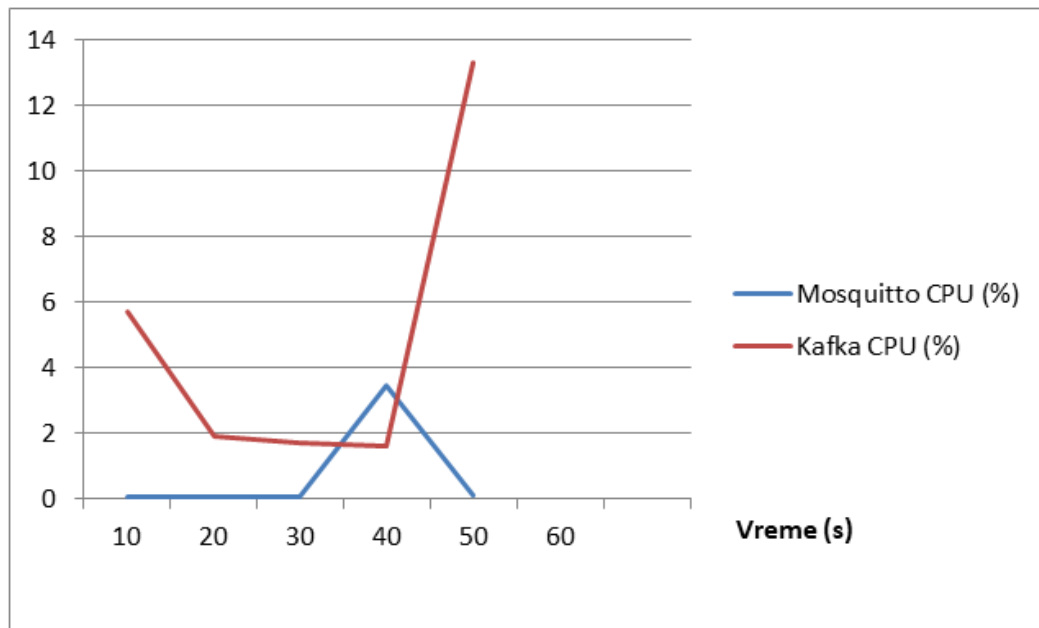
- **QoS 0 (At most once):** Omogucio je najmanju latenciju i najveći throughput, jer broker ne salje potvrde (fire-and-forget). Medjutim, ne nudi garanciju isporuke, što je dovodilo do potencijalnog gubitka poruka pri padu mreze.
- **QoS 1 (At least once):** Povecao je latenciju zbog obavezne PUBACK potvrde od strane brokera. Garantuje da ce poruka stici, ali moze doci do dupliranja poruka usled ponovnog slanja (retransmission).
- **QoS 2 (Exactly once):** Implementirao je slozeni four-way handshake, obezbedjujuci maksimalnu pouzdanost bez duplikata, ali je rezultovao najlosijim performansama i najvecom latencijom, cineci ga neadekvatnim za high-throughput sisteme.

Acks: Promena parametra acks=0 donela je maksimalni throughput jer producer ne ceka potvrdu. acks=1 nudi balans izmedju brzine i sigurnosti, dok acks=all drasticno obara performanse jer zahteva sinhronizaciju svih replika, ali nudi maksimalnu pouzdanost.

Consumer Lag: U Scenariju C primeceno je stvaranje zaostatka – broja poruka koje je producer poslao, a koje consumer-i jos nisu stigli da obrade. Kafka ovo resava kroz mehanizam particionisanja. Povećanjem broja particija u topic-u omogucena je masivna paralelizacija (dodavanjem vise instanci consumer-a u istu Consumer Group-u), cime je backlog efikasno i brzo rešen (recovery time).

Ovi mehanizmi su balans izmedju brzine i sigurnosti. U IoT sistemu gde je bitna brzina (real-time), koristimo nize QoS nivoe (0 ili 1) i acks=0 ili 1. Medjutim, kada nam je bitna tacnost (npr. alarmi za pozar), moramo da zrtvujemo brzinu i latenciju da bismo osigurali da se ni jedna poruka ne izgubi.

4. Scenario B (Edge Connectivity Failures)



Grafikon prikazuje Scenario B (pad i oporavak mreže). Tokom prve 30. sekunde prekid mreže uzrokuje minimalno CPU zauzeće kod oba brokera. U 40. sekundi dolazi do rekononekcije gde Mosquitto prvi reaguje sa pikom od 3,47%, dok Kafka u 50. sekundi doživljava skok od 13,30% CPU-a, što ukazuje na veći procesorski overhead koji Kafka zahteva prilikom obrade nagomilanih poruka nakon mreznog prekida.

1. Zasto je MQTT idealan za postavljanje na samim edge uređajima, a zasto postaje neadekvatan za istorijsku analitiku velikih podataka?

MQTT je idealan za edge okruženje jer koristi veoma malo sistemskih resursa. Kao što pokazuju podaci, Mosquitto je tokom maksimalnog opterećenja trosio jedva 23 MB RAM-a. Njegov header je velicine samo 2 bajta, što stedi mrežni protok na nestabilnim IoT mrežama. Međutim, on postaje neadekvatan za analitiku velikih podataka jer nije dizajniran kao baza. MQTT poruke nestaju čim se isporuče pretplatnicima, ne čuvaju se trajno na disku i broker ne nudi napredne koncepte poput replay-a istorijskih događaja ili paralelnog particionisanja za analitičke stream processing alate.

2. Zasto Kafka dominira u data-intensive cloud sistemima, kolika je "cena" njene skalabilnosti i da li je realno pokretati je na edge serverima?

Kafka dominira jer funkcioniše kao distribuirani append-only log, omogućavajući trajno skladištenje poruka, ponovno citanje istorije (offset-i) i masivnu paralelizaciju kroz particije.. Ipak, cena te skalabilnosti je ogromna potrošnja hardvera. U našem slučaju, Kafka je povukla preko 940 MB RAM-a i prešla 600% CPU zauzeca. Pokretanje Kafke na hardverski ograničenim edge uređajima je krajnje nerealno i neefikasno zbog ovih zahteva. Najbolje funkcioniše u Cloud okruženju.

3. Zaključak

Za robustan IoT ekosistem, optimalan pristup je hibridna arhitektura – korišćenje MQTT-a za komunikaciju na edge-u, uz integraciju sa Kafkom u Cloud-u kao centralnim hub-om za skladištenje, analitiku i dugoročnu obradu senzorskih podataka.